**МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ**
**РОССИЙСКОЙ ФЕДЕРАЦИИ**

Федеральное государственное автономное образовательное учреждение высшего образования
**«Московский авиационный институт**
**(национальный исследовательский университет)»**

# ОТЧЁТ ПО ЛАБОРАТОРНОЙ РАБОТЕ № 3

**Дисциплина:**

Операционные системы и архитектура компьютера

**Тема:**

Многопоточная битоническая сортировка массива целых чисел

**Выполнил:**

студент группы М8О-201БВ-25

Пантюхин Никита Олегович

**Проверил:**

Преподаватель

Миронов Евгений Сергеевич

Москва, 2026 г.

## Содержание

**1 Цель работы** **3**

**2 Краткое описание архитектуры решения** **3**

2.1 Схема взаимодействия компонентов . . . . . 4

**3 Листинг ключевых фрагментов кода** **5**

**4 Результаты работы** **8**

4.1 Консольный вывод и проверка потоков . . . . . 8

4.2 Ускорение и эффективность . . . . . 9

4.3 Влияние размера входного массива . . . . . 10

4.4 Влияние типа входных данных . . . . . 12

4.5 Скриншоты результатов . . . . . 13

**5 Выводы** **14**

## 1. Цель работы

Цель работы — получить практические навыки создания и синхронизации потоков в Linux. Необходимо реализовать на языке C многопоточную битоническую сортировку массива целых чисел, предусмотреть ограничение максимального количества потоков ключом запуска и продемонстрировать их фактическое количество средствами операционной системы. Также требуется исследовать зависимость ускорения и эффективности алгоритма от количества потоков, размера и типа входных данных.

## 2. Краткое описание архитектуры решения

Программа реализована на языке C с использованием библиотеки POSIX Threads (`pthread`). Проект собирается с помощью CMake. Вся реализация находится в одном исходном файле `main.c`: программа обрабатывает аргументы командной строки, создаёт массив, запускает рабочие потоки, измеряет время сортировки и проверяет порядок элементов.

Параметр `-t` задаёт максимальное количество одновременно участвующих потоков, включая главный. Параметр `-n` задаёт количество элементов, а необязательный параметр `-d` — тип входных данных: `random`, `sorted`, `reverse` или `repeated`. Размер массива ограничен степенями двойки, начиная с двух элементов. Число используемых потоков не превышает $n/2$, поскольку на каждом шаге сети выполняется $n/2$ независимых сравнений.

Битоническая сортировка последовательно строит группы элементов размера `size` и сравнивает пары на расстоянии `distance`. На каждом этапе потоки получают непересекающиеся диапазоны пар, поэтому не изменяют одни и те же элементы одновременно. После этапа все потоки встречаются на барьере `pthread_barrier_wait()`, чтобы следующий этап начался только после завершения текущего. Массив находится в общей памяти процесса, отдельные процессы не создаются.

## 2.1. Схема взаимодействия компонентов

На рисунке показано распределение работы: главный поток также участвует в сортировке, остальные потоки создаются функцией `pthread_create()`.

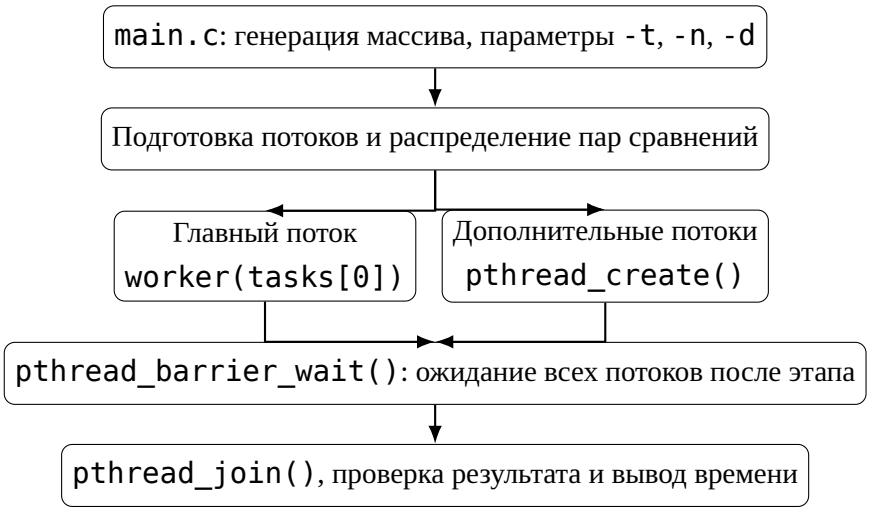



Рисунок — Схема работы многопоточной сортировки

На каждом этапе выполняются независимые сравнения, а барьер обеспечивает правильный порядок этапов. Внутри этапа синхронизация между отдельными сравнениями не требуется.

## 3. Листинг ключевых фрагментов кода

Основная работа выполняется функцией `worker()`. Внешний цикл увеличивает размер группы вдвое, внутренний цикл уменьшает расстояние между сравниваемыми элементами вдвое. Направление сравнения чередуется между группами.

```
void *worker(void *arg) {
    ThreadArgs *task = arg;

    for (long long size = 2; size <= task->n; size *= 2) {
        for (int distance = (int)(size / 2);
             distance > 0; distance /= 2) {
            for (int k = task->start; k < task->end; k++) {
                int i = (k / distance) * (2 * distance)
                        + (k % distance);
                int j = i + distance;
                int ascending = (i / size) % 2 == 0;

                if ((ascending && task->arr[i] > task->arr[j]) ||
                    (!ascending && task->arr[i] < task->arr[j])) {
                    int temp = task->arr[i];
                    task->arr[i] = task->arr[j];
                    task->arr[j] = temp;
                }
            }
            pthread_barrier_wait(task->barrier);
        }
    }
    return NULL;
}
```

Листинг 1 — Сравнение элементов в рабочем потоке

Каждому потоку передаются указатель на общий массив, размер массива, границы диапазона пар и указатель на барьер. Диапазоны формируются так, чтобы все $n/2$ пар были распределены между потоками без пересечений.

```
for (int t = 0; t < thread_count; t++) {
    tasks[t].arr = arr;
    tasks[t].n = n;
    tasks[t].start = (int)((size_t)t * comparisons / thread_count);
    tasks[t].end = (int)((size_t)(t + 1) * comparisons / thread_count
   );
    tasks[t].barrier = &barrier;
}
```

Листинг 2 — Распределение диапазонов между потоками

Дополнительные потоки создаются с помощью `pthread_create()`. Главный поток выполняет ту же функцию `worker()`, затем ожидает остальные потоки через `pthread_join()`.

```
for (int t = 1; t < thread_count; t++) {
    if (pthread_create(&threads[t - 1], NULL,
                       worker, &tasks[t]) != 0) {
        fprintf(stderr, "Thread creation failed\n");
        exit(EXIT_FAILURE);
    }
}

worker(&tasks[0]);

for (int t = 0; t < thread_count - 1; t++) {
    pthread_join(threads[t], NULL);
}
```

Листинг 3 — Создание потоков и ожидание их завершения

Время измеряется монотонными часами `CLOCK_MONOTONIC`: начало фиксируется перед созданием дополнительных потоков, а конец — после их завершения. Генерация массива, проверка результата и вывод не входят в измеренный промежуток. При этом учитываются создание, синхронизация и завершение потоков.

```
struct timespec start_time, end_time;
clock_gettime(CLOCK_MONOTONIC, &start_time);
```

Листинг 4 — Начало измерения времени

После выполнения функции `worker()` главным потоком и ожидания остальных потоков с помощью `pthread_join()` фиксируется конечное время и рассчитывается разность.

```
clock_gettime(CLOCK_MONOTONIC, &end_time);

double elapsed = (end_time.tv_sec - start_time.tv_sec) +
                 (end_time.tv_nsec - start_time.tv_nsec) / 1e9;
```

Листинг 5 — Окончание измерения времени

После сортировки программа последовательно проверяет, что каждый элемент не меньше предыдущего, и выводит `Sorted: yes` при выполнении этого условия.

```
int sorted = 1;
for (int i = 1; i < n; i++) {
    if (arr[i - 1] > arr[i]) {
        sorted = 0;
        break;
    }
}
```

Листинг 6 — Проверка порядка элементов после сортировки

Полный исходный код программы находится в файле `lab3/src/main.c`.

## 4. Результаты работы

Программа была собрана и проверена в среде WSL (Linux). Для исследования использовались массивы целых чисел размерами от 65 536 до 4 194 304 элементов, а также 1, 2, 4 и 6 потоков. При создании случайных массивов использовалось фиксированное начальное значение генератора `srand(12345)`, что обеспечивает одинаковый набор данных при повторении запуска с тем же типом и размером массива.

### 4.1. Консольный вывод и проверка потоков

Ниже приведён фактический результат одного из запусков программы с шестью потоками. Время приведено по выводу терминала.

```
$ ./build/lab3 -t 6 -n 1048576 -d repeated
Array size: 1048576
Requested threads: 6
Used threads: 6
Data type: repeated
Sorted: yes
Time: 0.294284 s
```

Листинг 7 — Пример успешной сортировки массива

Фактическое количество потоков было продемонстрировано стандартной командой Linux `ps -L`. В момент работы программы её вывод содержал шесть строк с одним PID и шестью разными TID.

```
    PID     TID STAT COMMAND
  75244   75244 Rl+  lab3
  75244   75254 Rl+  lab3
  75244   75255 Rl+  lab3
  75244   75256 Rl+  lab3
  75244   75257 Rl+  lab3
  75244   75258 Rl+  lab3
```

Листинг 8 — Потоки процесса по данным команды ps -L

Одинаковый PID подтверждает принадлежность потоков одному процессу, а разные значения TID — существование отдельных потоков. Таким образом, при ключе `-t 6` фактически использовались шесть потоков, включая главный.

### 4.2. Ускорение и эффективность

Ускорение $S(p)$ определялось как отношение времени однопоточной сортировки к времени работы $p$ потоков. Эффективность $E(p)$ показывает, насколько полно используется доступный параллелизм:

$$S(p) = \frac{T(1)}{T(p)}, \qquad E(p) = \frac{S(p)}{p} \cdot 100\%.$$

Для массива из 1 048 576 случайных элементов каждую конфигурацию запускали пять раз. В таблице приведены все фактические времена в секундах.

| Запуск | 1 поток | 2 потока | 4 потока | 6 потоков |
|---|---|---|---|---|
| 1 | 0,994904 | 0,474835 | 0,250886 | 0,336671 |
| 2 | 0,972518 | 0,498422 | 0,267290 | 0,313168 |
| 3 | 0,794891 | 0,426889 | 0,289247 | 0,330345 |
| 4 | 0,837853 | 0,548274 | 0,287952 | 0,320280 |
| 5 | 0,997526 | 0,537042 | 0,386954 | 0,327392 |

Таблица — Результаты пяти повторных измерений, с

Чтобы уменьшить влияние разовых колебаний, для каждого количества потоков выбрана медиана пяти измерений. Для ускорения используется отношение медианного времени одного потока к медианному времени соответствующего числа потоков.

| Потоки | Медиана, с | Ускорение | Эффективность, % |
|---|---|---|---|
| 1 | 0,972518 | 1,00 | 100,00 |
| 2 | 0,498422 | 1,95 | 97,56 |
| 4 | 0,287952 | 3,38 | 84,43 |
| 6 | 0,327392 | 2,97 | 49,51 |

Таблица — Ускорение и эффективность по медианным временам

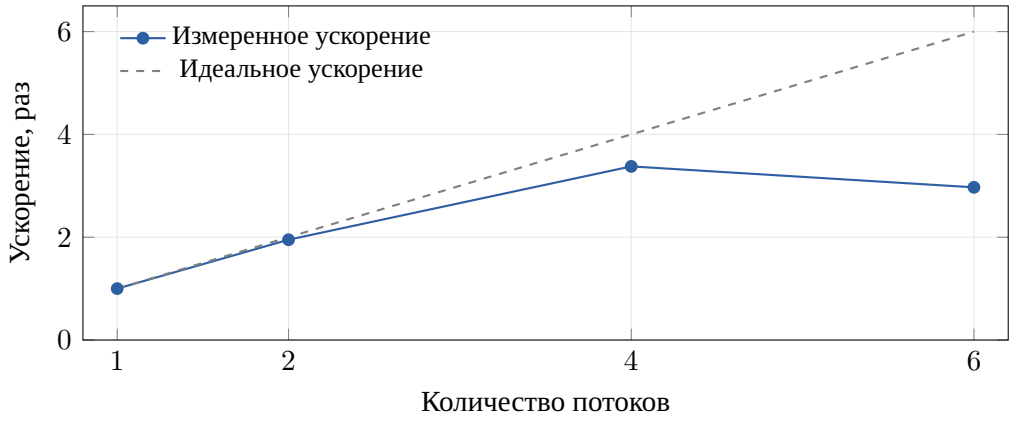



Рисунок — Зависимость ускорения от количества потоков

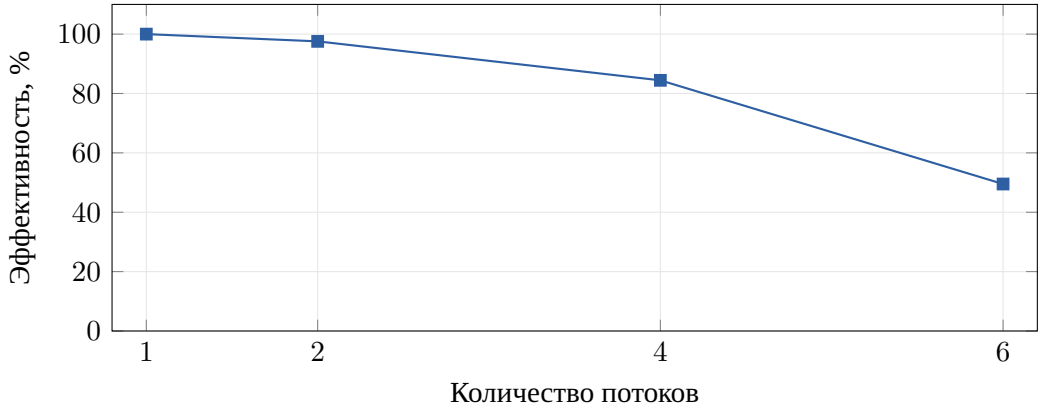



Рисунок — Зависимость эффективности от количества потоков

На данном размере массива наименьшее медианное время получено при четырёх потоках. Шесть потоков работали несколько медленнее: накладные расходы на создание и синхронизацию потоков, а также особенности выполнения в ОС могут снижать дополнительный выигрыш от параллелизма.

## 4.3. Влияние размера входного массива

Отдельно проведено по одному измерению для четырёх размеров случайного массива при каждом количестве потоков. Результаты приведены в секундах.

| Элементов | 1 поток | 2 потока | 4 потока | 6 потоков |
|---|---|---|---|---|
| 65 536 | 0,042258 | 0,029752 | 0,024685 | 0,024834 |
| 262 144 | 0,196706 | 0,125991 | 0,089934 | 0,083893 |
| 1 048 576 | 0,973696 | 0,475099 | 0,284199 | 0,339342 |
| 4 194 304 | 4,903357 | 1,966317 | 1,108901 | 1,102667 |

Таблица — Время сортировки в зависимости от размера массива, с

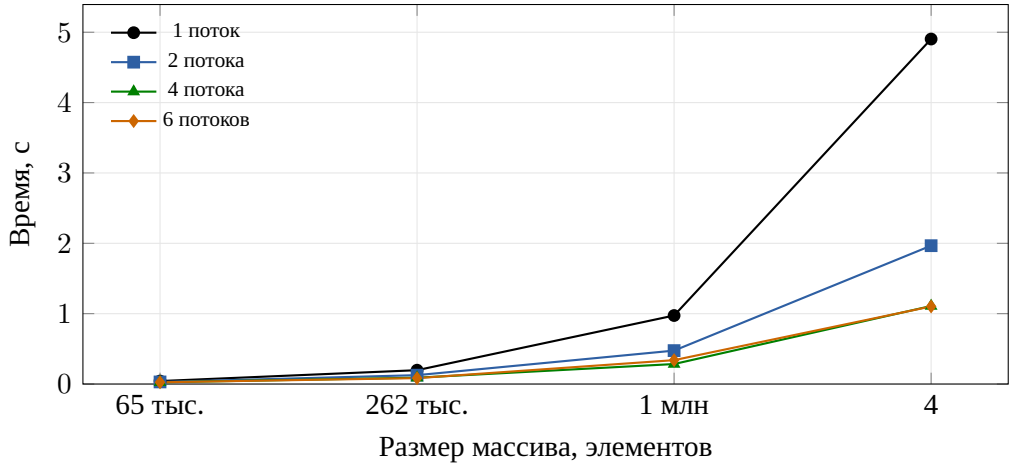



Рисунок — Зависимость времени сортировки от размера массива

В таблице ниже для каждого размера представлены ускорение и эффективность шести потоков относительно одного, рассчитанные по соответствующим одиночным измерениям.

| Элементов | Ускорение, 6 потоков | Эффективность, % |
|---|---|---|
| 65 536 | 1,70 | 28,36 |
| 262 144 | 2,34 | 39,08 |
| 1 048 576 | 2,87 | 47,82 |
| 4 194 304 | 4,45 | 74,11 |

Таблица — Ускорение и эффективность для различных размеров массивов

При увеличении массива с 65 536 до 4 194 304 элементов преимущество параллельной обработки становится заметнее. На наибольшем массиве один поток выполнял сортировку за 4,903357 с, а шесть потоков — за 1,102667 с, то есть в

данном запуске ускорение составило около 4,45 раза. Для небольшого массива затраты на работу с потоками составляют более значимую долю общего времени.

### 4.4. Влияние типа входных данных

Для массива из 1 048 576 элементов дополнительно сравнивались предварительно отсортированные, обратно отсортированные и содержащие повторения данные. Для каждой конфигурации выполнено по одному запуску.

| Тип массива | 1 поток | 2 потока | 4 потока | 6 потоков |
|---|---|---|---|---|
| `sorted` | 0,996246 | 0,405243 | 0,380148 | 0,312504 |
| `reverse` | 0,709495 | 0,418982 | 0,444729 | 0,467639 |
| `repeated` | 1,324132 | 1,406348 | 0,852322 | 0,624758 |

Таблица — Время сортировки различных входных данных, с

Для оценки влияния типа входных данных на масштабирование также рассчитаны ускорение и эффективность шести потоков относительно одного. Эти оценки основаны на одиночных измерениях.

| Тип массива | Ускорение, 6 потоков | Эффективность, % |
|---|---|---|
| `sorted` | 3,19 | 53,13 |
| `reverse` | 1,52 | 25,29 |
| `repeated` | 2,12 | 35,32 |

Таблица — Масштабирование сортировки для разных типов данных

В битонической сортировке последовательность этапов и число сравнений определяются размером массива, а не исходным порядком чисел. Различия наблюдаемого времени могут зависеть от числа обменов, поведения процессора, синхронизации и текущей нагрузки системы. Так как эти конфигурации измерялись однократно, результаты показывают наблюдения, но не доказывают устойчивое преимущество какого-либо типа массива или числа потоков.

## 4.5. Скриншоты результатов

В разделе предусмотрены места для фактических скриншотов терминала. Файлы `run.png`, `threads.png` и `measurements.png` необходимо положить в папку `report/screenshots/`; при следующей сборке они автоматически появятся на своих местах.




Рисунок — Успешный запуск многопоточной сортировки

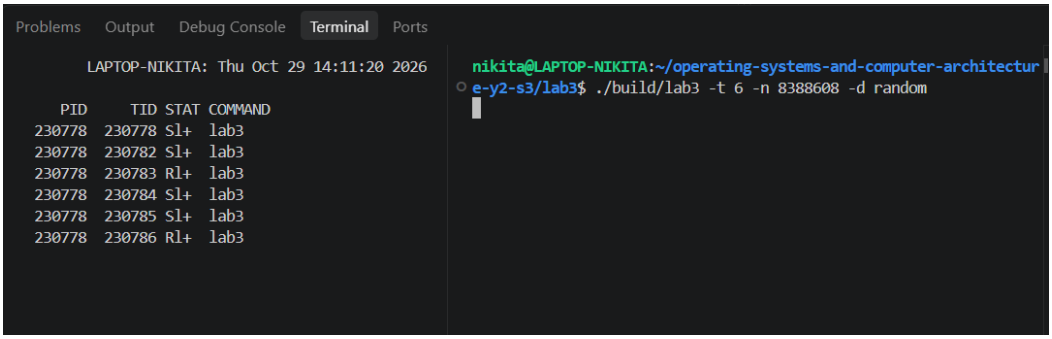



Рисунок — Наблюдение шести потоков с помощью команды ps -L

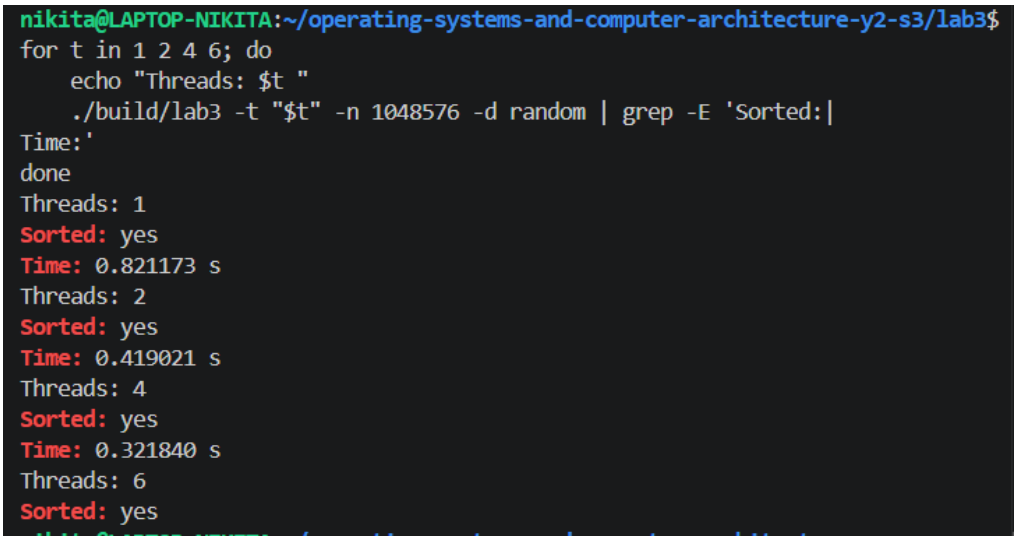



Рисунок — Измерения времени сортировки при разном количестве потоков

## 5. Выводы

После выполнения этой лабораторной я лучше понял, как устроена многопоточность в Linux и чем потоки отличаются от отдельных процессов. На практике я разобрался, как создавать потоки с помощью `pthread_create()`, ожидать их завершения через `pthread_join()` и синхронизировать работу с помощью барьеров.

При реализации битонической сортировки я научился распределять вычисления между несколькими потоками и проверять их количество средствами операционной системы. Также я увидел, что увеличение числа потоков не всегда приводит к ускорению программы, поскольку на производительность влияют размер массива и затраты на синхронизацию.

Благодаря этой работе я стал лучше понимать принципы параллельного выполнения задач и способы оценки его эффективности.